

RoboFollow: Unveiling the Instruction Following Mirage in Embodied Agents

Chang Guo¹ Yukun Xie¹ Bohan Tan¹ Zheng Chang¹ Zhaokai Yin¹
 Qianli Ma¹ Yingqiao Wang¹ Chao Liang² Zhipeng Zhang^{1,†}

¹AutoLab, School of Artificial Intelligence, Shanghai Jiao Tong University

²Research Lab, Anyverse Dynamics

zhipengzhang@sjtu.edu.cn

Abstract: Modern embodied agents achieve impressive success rates, yet their actual instruction-following ability is far weaker than these numbers suggest. We trace this illusion to a structural property we term low *scene entropy*: when a visual scene admits only one valid task, language becomes redundant and a policy can score highly while barely using it. We introduce **RoboFollow**, a diagnostic benchmark with three principles: (1) **High Scene Entropy**: each training scene supports multiple kinematically distinct task branches, making vision alone insufficient and forcing reliance on language. (2) **Hierarchical Diagnostic Protocol**: a four-level protocol (L0–L3) progressively perturbs visual layout and semantics, probing whether equivalent instructions yield consistent behavior and distinct ones yield discriminable behavior across spatial relations, attributes, trajectory constraints, and logic. (3) **Confound-Controlled Diagnosis**: we simplify interaction objects, restrict actions to the trained repertoire and report stage-wise Intent and Execution scores, isolating comprehension from motor execution. Evaluation of nine VLA and WAM policies shows that strong L0 performance, where attained, does not reliably transfer to L1–L3 under our fine-tuning setup. Representative mitigations, including stronger VLM backbones, QA co-training, LangForce, and Classifier-Free Guidance, all fail to close this gap. RoboFollow exposes genuine instruction following as a critical, overlooked bottleneck. Code and dataset are available at <https://github.com/AutoLab-SAI-SJTU/RoboFollow> and <https://huggingface.co/datasets/AutoLab-SJTU/robofollow-data>.

Keywords: Embodied Artificial Intelligence, Vision-Language-Action Models, World Action Models, Instruction Following

1 Introduction

Language-conditioned robot policies are increasingly expected to serve as general-purpose embodied agents. Recent Vision-Language-Action (VLA) [1, 2, 3] and World Action Models (WAM) [4, 5] have achieved strong success rates on manipulation benchmarks, while modular embodied systems increasingly use language as the interface between human intent and physical execution [6]. However, high task success does not necessarily imply instruction following. A policy may complete a task because the visual scene already suggests a plausible action, while the language instruction is ignored, weakly used, or treated merely as a task identifier. For deployable robots, this distinction is critical: a visually successful action can still be semantically wrong.

This problem is difficult to expose with standard evaluation protocols. Most manipulation benchmarks emphasize final task success, which entangles visual recognition, language grounding, planning, and low-level control. More importantly, many benchmark episodes contain a dominant or

[†]Corresponding Author.

canonical behavior that can be inferred from the initial visual observation, making language partially redundant. Such benchmarks remain valuable for measuring manipulation competence, but they are less diagnostic of whether a policy uses language to choose among multiple feasible behaviors. Recent studies further show that VLA policies can be insensitive to linguistic perturbations on existing benchmarks [7, 8, 9]. RoboFollow complements these evaluations by constructing task ambiguity during training and diagnosing grounding through controlled splits and stage-wise scoring.

To address this gap, we introduce **RoboFollow**, a diagnostic benchmark for evaluating whether embodied agents use language to select and execute the intended behavior. RoboFollow is built around a simple principle: language should be necessary rather than optional. It constructs high-ambiguity scenes in which the same or highly similar visual configuration supports multiple semantically valid and kinematically feasible task branches. In such scenes, vision alone is insufficient to identify the intended behavior, forcing the policy to rely on the instruction for disambiguation. We quantify this ambiguity through *scene entropy*, the conditional entropy of training task labels given the task-independent initial scene specification.

RoboFollow evaluates instruction following along three complementary dimensions. First, it covers four scene families: spatial relations, object attributes and action selection, trajectory and orientation constraints, and elementary logical grounding. Second, it introduces a four-level evaluation protocol: L0 measures in-distribution performance, L1 tests visual grounding under changed layouts, L2 tests semantic recombination under fixed layouts, and L3 combines visual and semantic perturbations. Third, it separates semantic misunderstanding from motor failure through stage-wise *Intent* and *Execution* Scores. Intent measures whether the policy selects the correct object, relation, waypoint, or logical branch, while Execution measures whether the selected subgoal is physically completed.

We conduct a systematic evaluation of representative VLA and WAM models on RoboFollow. Across models and scene families, we observe a consistent pattern: models achieve strong in-distribution performance at L0, but their Intent Scores degrade sharply under L1–L3. We further examine stronger VLM backbones, QA co-training, LangForce, and classifier-free guidance, but none closes the generalization gap beyond L0. These results suggest that robust language-conditioned task selection remains a challenge for the evaluated policies after fine-tuning.

Our contributions can be summarized as follows:

1. **A language-necessary diagnostic benchmark with controlled execution confounds.** We introduce RoboFollow, which constructs high-entropy scenes where language is required to disambiguate among multiple feasible task branches. By using simple objects, short-horizon interactions, and action primitives covered by training, RoboFollow reduces motor-execution confounds and enables a targeted diagnosis of semantic instruction following.
2. **A hierarchical diagnostic protocol with stage-wise scoring.** We design L0–L3 evaluation splits that progressively test in-distribution execution, visual grounding under layout changes, semantic recombination under familiar visual contexts, and joint visual-semantic generalization. We further report stage-wise Intent and Execution Scores to separate errors in intent selection from execution inaccuracies conditioned on a correct intent.
3. **A systematic analysis of current models and optimizations.** We evaluate representative VLA and WA models, together with stronger VLM backbones, QA co-training, LangForce, and classifier-free guidance, and show that these strategies remain insufficient for robust instruction following.

2 Related Work

2.1 Vision-Language-Action and World Action Models

Vision-Language-Action (VLA) models [1, 2, 3, 10, 11, 12] extend pre-trained VLMs to connect internet-scale perception with physical execution across diverse datasets [13]. Early methods discretize continuous actions for autoregressive generation [14, 1, 15, 2], whereas recent ar-

chitectures improve precision through flow matching [3, 16, 10], physically grounded representations [17, 18, 19], efficient quantization [20, 21], and visual chain-of-thought reasoning [11, 22, 12]. World Action Models (WAM) move beyond reactive visuomotor mapping by coupling action generation with predictive modeling of environment dynamics. This paradigm advances from diffusion-based visuomotor policies [23, 24, 25] to predictive world models that jointly model visual dynamics and actions, using video generation to forecast future states and infer the corresponding actions [26, 27, 28, 29, 5, 30, 31].

2.2 Benchmarks for Robotic Manipulation Evaluation

Robotic manipulation benchmarks span simulation and real-world evaluation. RL Bench [32] and SimplerEnv [33] provide standardized control protocols, while CALVIN [34], VIMA [35], VLABench [36], and RoboTwin2.0 [37] target long-horizon interaction, multimodal reasoning, semantic generalization, and scalable demonstration generation [36, 38, 37, 39, 40]. LIBERO [41], LIBERO-PRO [8], and LIBERO-Plus [9] further study knowledge transfer and robustness, showing that policies often exploit visual patterns instead of language semantics. RoboFollow explicitly pairs shared training scenes with multiple task specifications. This construction addresses training-time task ambiguity, complementing the test-time perturbations studied in LIBERO-PRO. $\pi_{0.7}$ reports stronger instruction following with more diverse data and richer contextual conditioning [42]; these different training and evaluation settings motivate our diagnostic. IVA addresses false-premise verification and correction [43], complementing RoboFollow’s selection among feasible tasks.

3 RoboFollow Benchmark

RoboFollow is designed to evaluate whether embodied agents use language to select and execute the intended behavior, rather than relying on visual shortcuts. Its central design principle is to make language necessary for task identification. In each diagnostic scene, the same or highly similar visual configuration supports multiple semantically valid and kinematically feasible task branches. Therefore, the initial observation alone is insufficient to infer the target behavior, and the policy must condition on the instruction to resolve task ambiguity.

Concretely, RoboFollow contains four diagnostic scene families comprising 75 training task labels grouped into six task-independent initial scene configurations. Let T denote a training task label (shared by its paraphrases) and S the task-independent initial scene specification, including objects, placement distributions, and observable state predicates, excluding instructions and goals. We define

$$H_{\text{scene}} = H_{\text{train}}(T \mid S) = - \sum_s p(s) \sum_t p(t \mid s) \log_2 p(t \mid s). \quad (1)$$

Here, $p(s)$ is the fraction of training demonstrations associated with scene specification s , and $p(t \mid s)$ is the fraction of demonstrations within that scene group labeled with task t .

With equal demonstrations per task, $H_{\text{scene}} = \sum_s (K_s/K) \log_2 K_s$, where K_s counts task labels sharing scene specification s and $K = \sum_s K_s$. RoboFollow has $K_s = (16, 16, 16, 16, 7, 4)$ and $K = 75$, yielding 3.782 bits, compared with 0.880 bits for the equally weighted LIBERO Spatial/Object/Goal/Long suites under the same task-label definition. Grouping details are given in Appendix A.1. We use scene entropy as a dataset design principle rather than as a final evaluation metric: its role is to remove visual shortcuts during training and evaluation. After this ambiguity is established, RoboFollow diagnoses instruction following through controlled test splits and stage-wise Intent and Execution scores.

3.1 Scene Design

RoboFollow evaluates models with four hierarchical test levels. These levels are not intended as a strict difficulty ordering; instead, they isolate different generalization dimensions. Overview of RoboFollow scene design is shown in Fig. 1, more details are provided in Appendix A.

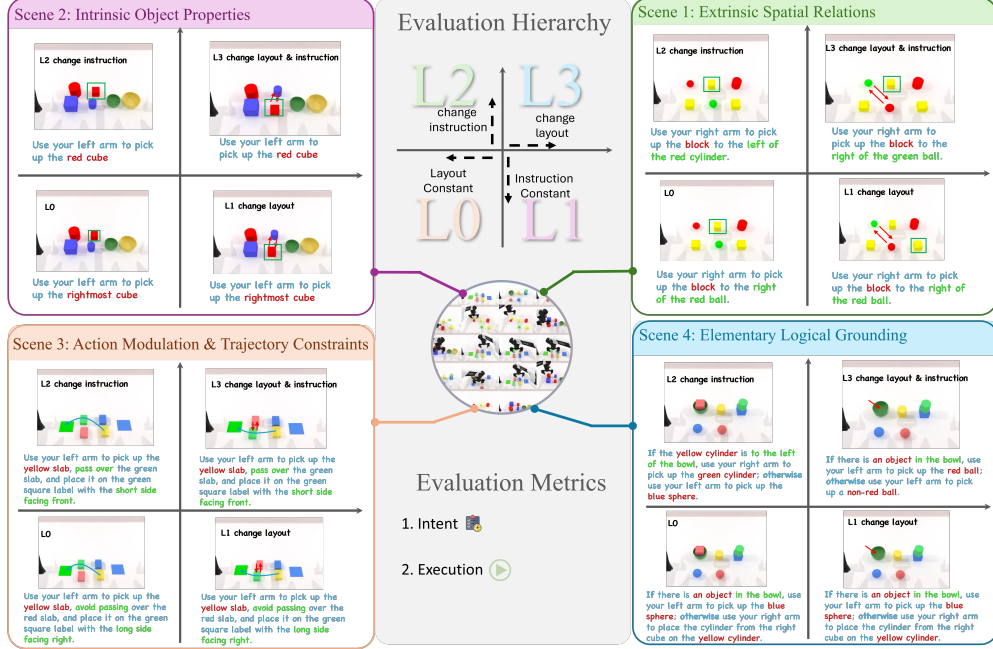


Figure 1: **Overview of RoboFollow scene design.** RoboFollow contains four diagnostic scenes targeting extrinsic spatial relations, intrinsic object properties, action and trajectory constraints, and elementary logical grounding. Each scene is evaluated under four levels, from in-distribution testing to combined visual-semantic perturbations.

♥ **L0 (In Distribution):** The test configuration follows the training distribution and measures standard in-distribution performance.

♥ **L1 (Visual Grounding):** The instruction remains unchanged, but object positions are swapped. This tests whether the model binds language to the correct physical entities rather than memorizing spatial coordinates.

♥ **L2 (Semantic Compositionality):** The visual layout remains unchanged, but the instruction uses novel recombinations of semantic attributes seen during training. This tests whether the model captures atomic meanings rather than memorizing specific text–object or attribute–object pairings.

♥ **L3 (Visual-Semantic Mixture):** Both the visual layout and the instruction are perturbed, evaluating whether the model can jointly handle visual grounding and semantic recombination.

All training and validation splits are constructed to preserve linguistic clarity, avoid leakage across evaluation levels, and ensure that test-time target actions remain within the demonstrated behavioral repertoire. We next provide a concise overview of the four diagnostic scenes.

Scene 1: Extrinsic Spatial Relations. Scene 1 evaluates whether agents can ground extrinsic spatial relations such as “to the left of”, “to the right of”, and “behind”. The core challenge is that multiple candidate objects are visually similar or identical, so the target cannot be determined from appearance alone. Instead, the policy must identify the correct object by binding relational language to the current spatial configuration. This scene therefore tests whether the model truly understands spatial prepositions, rather than memorizing absolute object positions or canonical layouts.

Scene 2: Intrinsic Object Properties. Scene 2 evaluates compositional grounding over intrinsic object properties and action primitives. Instructions specify combinations of attributes such as color, size, and shape, together with actions such as pick, push, stack, and place. The object sets are designed so that no single attribute is always sufficient for identifying the target, requiring the model to compose multiple semantic cues. When necessary, kinematic choices such as arm selection are made explicit in the instruction, reducing ambiguity from physical feasibility.

Scene 3: Fine-Grained Action Modulation. Scene 3 evaluates whether models can follow procedural constraints beyond achieving a final goal state. In this scene, instructions specify not only the source object and destination, but also intermediate motion constraints such as waypoints and final orientations. This design tests whether the policy follows the instructed procedure, rather than merely producing an action that ends in a plausible final configuration. It is particularly useful for diagnosing whether models treat language as a coarse task label or as a fine-grained control signal.

Scene 4: Elementary Logical Grounding. Scene 4 evaluates elementary logical forms required for robust instruction following, including temporal sequencing, explicit negation, and observable conditional branching. Unlike long-horizon planning benchmarks, this scene focuses on short and controlled instructions such as “first do A then do B”, “not A”, and “if A then do B else do C”. The relevant scene state is varied across splits, so the policy must evaluate the current observation rather than memorizing fixed instruction–trajectory pairs.

3.2 Metrics

Binary task success based on the final environment state is often insufficient for diagnosing instruction following. It may penalize a policy that selects the correct intent but fails due to a minor execution error, while rewarding a policy that reaches the final state after semantically incorrect intermediate actions. RoboFollow therefore adopts a **Multi-Stage Intent-Execution Scoring** framework, where each task is decomposed into sequential sub-stages and evaluated using two complementary scores. More details are provided in Appendix B.

♥ **Intent Score (Semantic Grounding):** Intent Score measures whether the policy selects the correct semantic target at each stage, such as the intended object, relation, waypoint, action primitive, orientation, or logical branch. It focuses on whether the policy attempts the behavior specified by the instruction, even when physical execution is imperfect.

♥ **Execution Score (Kinematic Proficiency):** Execution Score measures whether the corresponding subgoal is successfully completed, such as grasping, transporting, or placing the selected object.

By separating intent from execution, RoboFollow distinguishes semantic misunderstanding from low-level control failure and penalizes policies that achieve the final state through incorrect intermediate actions. A dedicated finishing stage accounts for 20% of the score, requiring policies to refrain from further actions unrelated to the instruction after completing the requested task in order to receive full credit.

Completion Rate (CR) complements IS and ES with an action-dependent binary completion criterion. Within each scene and difficulty level, CR is the unweighted mean of per-task completion rates. It does not require every intermediate stage to receive full credit. The exact signal priority and temporal scope are specified in Appendix B.

4 Experiments and Results

In this section, we deploy the RoboFollow benchmark to systematically diagnose the instruction following capabilities of state of the art embodied agents. Rather than merely reporting success rates, we structure our evaluation around several core research questions designed to unmask the illusion of competence.

4.1 Experimental Setup

Rather than exhaustively evaluating lower-capacity baselines, we select nine state-of-the-art foundation models spanning the dominant VLA and WAM paradigms. These large-scale models feature extensive pre-training, strong semantic reasoning, and broad community adoption, allowing us to study instruction following across representative pre-trained policies without isolating architecture from data or optimization effects. Our VLA evaluation includes π_0 [3], its successor $\pi_{0.5}$ [11], NVIDIA’s GR00T N1.6 [10], openvla-ofit [44], xvla [45], ACoT-VLA [46], and Lingbot-VLA [47].

Table 1: Performance comparison across all RoboFollow scenes. All metrics are reported as percentages (%). For each scene and difficulty level, the best score per metric across all models is in **bold**. The Avg block reports each model’s mean IS, ES, and CR over the four scenes (S1–S4) and all difficulty levels; the best model per Avg metric is bolded.

Model	Scene	L0			L1			L2			L3			Avg		
		IS	ES	CR	IS	ES	CR	IS	ES	CR	IS	ES	CR	IS	ES	CR
PI05	S1	99.1	98.5	98.8	45.5	43.8	42.5	58.0	58.0	58.1	44.9	43.8	40.1	55.7	53.0	47.3
	S2	100.0	100.0	100.0	77.5	69.4	68.8	32.4	24.6	20.6	34.2	26.7	16.7			
	S3	85.0	85.0	81.3	66.9	66.5	53.8	30.0	29.8	0.0	42.5	42.5	12.5			
	S4	84.1	78.1	79.6	34.0	31.3	33.3	33.6	30.4	30.0	22.9	20.4	20.8			
PI05 empty language	S1	16.0	16.0	6.3	0.5	0.5	0.0	16.6	16.6	6.3	17.4	16.6	6.7	12.3	8.9	5.2
	S2	16.6	9.4	9.4	18.5	10.6	9.6	23.1	14.0	11.2	19.3	13.9	12.9			
	S3	6.4	2.1	0.0	8.2	4.1	0.0	6.5	2.0	0.0	7.5	2.8	0.0			
	S4	20.4	16.5	8.2	4.7	4.3	3.3	11.0	10.4	6.4	3.7	2.4	2.5			
PI0	S1	98.2	98.2	97.5	0.0	0.0	0.0	4.1	0.0	0.0	15.7	0.8	0.9	48.4	44.9	38.9
	S2	95.6	95.3	95.6	77.5	67.1	66.3	36.3	27.4	21.9	35.2	29.1	18.8			
	S3	96.3	96.3	95.0	69.8	69.8	58.0	29.7	29.7	0.2	45.0	45.0	15.3			
	S4	84.1	81.9	81.9	26.5	23.9	20.0	36.6	33.1	32.8	23.2	20.4	17.5			
GROOT N1.6	S1	71.4	67.1	62.5	0.0	0.0	0.0	24.0	24.0	19.4	36.0	34.0	25.8	32.7	25.6	20.5
	S2	70.9	51.2	49.4	23.7	5.8	5.4	16.1	10.9	10.3	12.3	7.2	7.5			
	S3	56.4	52.3	48.8	33.9	30.5	19.8	23.7	17.0	0.0	20.0	14.4	0.3			
	S4	57.8	38.2	34.1	31.9	28.7	23.3	25.3	15.8	12.4	19.6	13.0	9.2			
OpenVLA-OFT	S1	9.8	9.5	6.3	0.0	0.0	0.0	2.2	2.2	0.0	2.6	1.8	1.1	10.7	4.2	3.5
	S2	28.9	10.3	10.6	23.3	2.1	2.5	10.2	4.3	3.8	18.9	10.3	9.2			
	S3	7.9	1.6	0.6	7.5	1.1	0.0	6.9	0.3	0.0	6.4	0.3	0.0			
	S4	23.5	9.6	7.8	9.3	6.7	6.7	5.0	0.0	0.0	8.6	7.7	7.7			
XVLA	S1	76.7	74.5	71.9	0.0	0.0	0.0	24.2	23.5	12.5	45.7	44.3	33.0	34.3	30.9	22.3
	S2	76.7	65.8	66.9	6.8	6.8	0.0	38.7	30.1	24.3	16.3	10.9	7.5			
	S3	74.0	69.9	45.6	22.5	22.2	5.0	33.0	32.9	0.9	34.8	33.0	4.4			
	S4	38.5	34.9	39.3	6.9	5.9	3.3	29.4	23.2	25.2	24.5	17.2	16.7			
ACoT-VLA	S1	100.0	100.0	100.0	8.7	6.2	6.3	60.9	57.2	59.4	48.0	45.5	42.1	53.5	49.3	42.5
	S2	96.4	91.6	89.4	76.3	53.4	52.5	40.0	33.5	26.6	34.0	25.0	18.3			
	S3	95.6	95.4	89.4	77.5	77.0	64.8	30.3	30.3	0.3	39.7	39.4	1.9			
	S4	78.7	72.8	72.6	23.6	23.6	23.3	26.2	22.0	18.4	20.3	15.2	15.4			
Lingbot-VLA	S1	80.9	78.4	82.5	0.0	0.0	0.0	24.2	23.7	23.8	46.4	43.5	37.2	35.2	32.5	27.4
	S2	77.1	67.6	74.4	7.0	5.8	0.0	32.5	28.0	23.5	6.6	4.1	1.7			
	S3	99.1	98.6	97.5	24.8	24.7	9.8	33.2	32.6	0.2	29.8	29.2	0.1			
	S4	58.0	47.1	51.1	8.9	6.3	7.3	18.4	16.8	18.4	16.3	14.2	10.8			
MOTUS	S1	73.4	68.8	70.6	11.3	11.1	13.8	26.9	26.1	33.1	29.8	26.7	27.0	36.8	30.8	27.3
	S2	78.2	69.9	81.3	53.1	43.6	50.8	17.9	13.6	8.2	16.2	11.0	8.3			
	S3	67.6	61.2	45.6	42.6	37.0	17.8	30.9	26.0	2.0	36.6	31.8	10.3			
	S4	51.7	34.9	36.3	11.6	4.0	1.3	20.6	13.1	15.2	20.6	14.4	15.8			
FASTWAM	S1	83.1	83.1	94.4	0.0	0.0	0.0	13.0	4.0	3.1	41.9	36.2	31.1	39.3	35.2	30.3
	S2	82.5	76.5	85.0	32.0	14.1	11.7	24.8	20.4	14.6	18.0	5.3	2.9			
	S3	98.3	98.1	95.6	60.4	59.7	42.0	30.6	30.5	0.3	45.0	43.3	4.9			
	S4	52.1	48.4	56.3	9.1	6.9	4.0	21.1	20.6	23.2	16.7	15.5	15.4			

IS: Intent Score; ES: Execution Score; CR: Completion Rate. Avg: per-model mean of each metric over the four scenes and all difficulty levels.

For WAM, we evaluate Motus [5] and FAST-WAM [48]. The four scenes comprise a total of 3,750 training episodes. Our main experiments were all fine-tuned using all the data, while the analysis part of the experiments only used 16 tasks (800 episodes) from scene2 for fine-tuning. Detailed training configurations are provided in the appendix.

4.2 Main Results

RQ1: Do Current Embodied Agents Genuinely Follow Instructions? *The short answer is no.* As shown in Tab. 1, while leading models project an illusion of competence under in-distribution conditions, their instruction following capabilities collapse precipitously once even minimal perturbations are introduced. Averaged across all scenes, all models show a pronounced decline in IS from L0 to L1–L3, the consistent drop in Intent Score indicates that the degradation cannot be explained solely by low-level execution failures; failures in semantic intent selection are a major contributing factor. On Scenes 1 and 2, which test spatial relation grounding and intrinsic attribute binding, the

π -series models achieve near-saturated L0 performance: $\pi_{0.5}$ obtains 99.1% and 100.0% IS, and π_0 reaches 98.2% and 95.6%. However, performance drops sharply at higher levels. In Scene 1, π_0 falls from 98.2% at L0 to **0.0%** at L1, while $\pi_{0.5}$ declines from 99.1% to 45.5% at L1 and 44.9% at L3. A similar pattern appears in Scene 2, where $\pi_{0.5}$ decreases from 100.0% at L0 to 34.2% at L3.

RQ2: Where Embodied Agents Break Down? As shown in Tab. 1, an analysis of cross-scene generalization reveals differences across evaluated policies in their generalization over distinct semantic dimensions. Specifically, π_0 , GR00T N1.6, and Motus fail precipitously on the L1 visual grounding evaluation of Scene 1, a pattern consistent with limited spatial grounding and reliance on learned scene-task associations. While $\pi_{0.5}$ exhibits comparatively stronger semantic grounding for these spatial relationships, its performance still degrades substantially under visual perturbations. Interestingly, models such as π_0 and Motus demonstrate significantly more robust semantic grounding when processing intrinsic object attributes in Scene 2 than they do with the extrinsic spatial relations in Scene 1. Failure cases are shown in Appendix E.

Real-robot pilot. We evaluate $\pi_{0.5}$ using eight training instructions and eight held-out instructions over the same object set. Each instruction is tested five times: success decreases from 20/40 (50%) on training instructions to 6/40 (15%) on held-out instructions. This preliminary comparison uses different instruction sets and pick/stack compositions, rather than matched task pairs; full instructions and counts appear in Appendix C.1.

5 Analysis

The severe performance degradation across generalization levels L1–L3 strongly suggests that existing models treat language instructions as shallow task identifiers rather than genuinely understanding their semantics and grounding them to target objects and actions. To investigate the mechanisms of these failures, we conduct an in-depth diagnostic analysis on Scene 2, examining deficiencies in the underlying VLM and evaluating whether recent mitigation strategies, such as stronger VLM backbones and QA co-training, can address this bottleneck.

5.1 VLM Deficits and The Comprehension-Execution Gap

A natural first question is whether instruction-following failures originate from the vision-language backbone itself or emerge downstream during action generation. We design a diagnostic protocol that independently probes (i) the VLM’s scene comprehension and (ii) the fidelity with which comprehended semantics propagate to action generation.

We constructed a visual QA probe targeting object identities, colors, and spatial relations in Scene 2. The complete set of 20 questions is provided in Appendix D. We evaluated the base PaliGemma, the pre-trained $\pi_{0.5}$ and the fine-tuned $\pi_{0.5}$ backbone. Scene understanding accuracy remained consistently low with scores of 1/20, 2/20, and 3/20, respectively.

Even for questions that the fine-tuned VLM answers correctly, about half of these episodes still result in incorrect manipulation behavior. This reveals a two-layered failure structure: the VLM backbone frequently lacks sufficient scene understanding, and even when comprehension succeeds, the action generation pipeline fails to faithfully translate it into behavior. Notably, in Motus, whose VLM backbone is entirely frozen throughout training, we observe a comparable disconnect between correct visual comprehension and successful manipulation execution. A natural follow-up question is whether these bottlenecks can be alleviated by (i) employing a substantially more powerful VLM backbone, and (ii) applying recently proposed optimization strategies specifically designed to preserve or enhance instruction-following capabilities.

5.2 Can Stronger VLMs and Existing Optimizations Help?

The analysis in Sec. 5.1 identifies two cascading bottlenecks: inadequate scene comprehension in the VLM backbone, and a comprehension-to-execution gap in the action head. We now ask whether

Table 2: Performance comparison on Scene 2 (all metrics in %).

Model	L0			L1			L2			L3		
	IS	ES	CR	IS	ES	CR	IS	ES	CR	IS	ES	CR
pi05	93.6	89.0	89.4	75.7	42.0	40.4	37.8	30.3	25.7	45.8	24.9	19.2
pi05_cfg.1.2	55.9	44.7	46.9	50.4	25.0	25.0	31.2	22.4	21.3	31.2	14.2	8.3
pi05_cfg.1.5	43.6	25.9	25.0	37.8	23.5	25.0	28.6	20.0	20.6	24.6	12.3	10.4
qwengroot	67.9	63.1	65.6	6.3	0.0	0.0	31.1	21.7	17.2	14.8	1.3	1.7
qwengroot_qa1	60.1	46.4	45.0	3.5	0.8	0.8	26.4	15.6	10.3	18.9	8.8	9.2
qwengroot_qa2	68.2	61.3	60.6	0.0	0.0	0.0	21.8	16.0	16.6	0.0	0.0	0.0
qwengroot_qa3	66.6	50.5	46.3	2.9	0.0	0.0	30.8	19.3	15.0	0.0	0.0	0.0
langforce	65.1	44.2	35.0	6.3	4.0	0.0	18.6	14.7	14.1	0.0	0.0	0.0

IS: Intent Score; ES: Execution Score; CR: Completion Rate.

All models are fine-tuned only on Scene 2 data.

In pi05-cfg-x, x denotes the CFG guidance scale.

qa1 uses ShareGPT4V-COCO QA, qa2 uses our Scene 2 QA dataset, and qa3 uses mixed QA.

three representative mitigation strategies can alleviate these failures: (i) upgrading the VLM backbone, (ii) QA co-training to preserve linguistic competence, and (iii) language-conditioned guidance. Results are summarized in Table 2.

Following the above observations, we evaluated Qwen3-VL (4B), a significantly more powerful vision-language backbone. On our fine-grained QA probe test set for Scene 2, Qwen3-VL-4B achieved an impressive success rate of 19/20, demonstrating highly robust zero-shot spatial and attribute comprehension.

♥ We trained Qwen-GR00T by integrating the Qwen3-VL-4B backbone with the GR00T diffusion action head, to mitigate the widely observed phenomenon of catastrophic forgetting, where action fine-tuning erases pre-trained linguistic capabilities, we implemented a QA co-training strategy. During this process, visual QA pairs and manipulation trajectories were jointly optimized. However, the out-of-distribution instruction following capabilities across L1–L3 still exhibit a catastrophic collapse in high-entropy scenarios. This indicates that merely possessing a stronger VLM and explicitly maintaining its QA capabilities during training is insufficient to bridge the comprehension to execution gap when complex physical grounding is required.

We further examined two strategies that aim to amplify linguistic influence over generated actions.

♥ *LangForce* [49], a recent method that strengthens text–action correlation during training, yields only marginal changes. We attribute this to the method’s underlying mechanism: while LangForce successfully amplifies the statistical correlation between the instruction text and the generated motion distribution, it lacks explicit supervision for genuine, compositional semantic understanding.

♥ *Classifier-Free Guidance* (CFG) [50], a standard technique for boosting conditioning signals in diffusion policies, proves counterproductive in our setting. Increasing the guidance scale to 1.2 and 1.5 degrades even L0 performance and produces erratic trajectories. In low-entropy benchmarks, the unconditional prediction provides a stable baseline because vision alone largely determines the task; in our high-entropy scenes, dropping the language input forces the model into multimodal guessing, rendering the CFG residual dominated by noise rather than a clean semantic signal.

None of the evaluated strategies, whether targeting the VLM backbone, the training objective, or the inference procedure, yields satisfactory generalization beyond L0 in high-entropy environments. These results leave instruction following unresolved in the tested configurations, without ruling out improvements from broader data distributions or alternative training procedures. These observations motivate increasing structural scene and semantic diversity while preserving held-out combinations, and supervising grounded language-to-action alignment during adaptation. These directions remain hypotheses for future evaluation.

5.3 Additional Fine-Tuning Controls

We examine whether the observed generalization gaps persist under changes to the fine-tuning setup. These controls use $\pi_{0.5}$ on the 16 Scene 2 tasks, separately from the full-benchmark experiment. Table 3 reports the Intent Scores under changes in demonstration count, instruction variants, and training duration.

Table 3: Scene 2 fine-tuning controls: Intent Score (%). Each panel varies the indicated factor. 1/3/5 denotes total instruction variants.

Factor	Value	L0	L1	L2	L3
Demonstrations/task	25	88.4	69.8	33.5	39.8
	50	93.6	75.7	37.8	45.8
Instruction variants	1	93.1	72.8	31.6	41.5
	3	93.6	75.7	37.8	45.8
	5	93.1	79.0	32.0	53.0
Training steps	2k	86.4	79.2	31.5	38.9
	4k	92.4	77.6	35.7	47.6
	6k	93.6	75.7	37.8	45.8

Doubling demonstrations from 25 to 50 per task improves all Intent Scores, but leaves large gaps: L0–L2 changes from 54.9 to 55.8 percentage points, and L0–L3 from 48.6 to 47.8.

Increasing instruction variants from one to five improves L1 (72.8% to 79.0%) and L3 (41.5% to 53.0%), while L2 varies non-monotonically (31.6%, 37.8%, 32.0%). From 2k to 6k steps, L0/L2 improve and L1 decreases; L3 peaks at 4k. At 6k, L0–L2/L3 gaps remain 55.8/47.8 points.

Within the tested ranges, these controls improve $\pi_{0.5}$ on Scene 2 but leave substantial L0–L2/L3 gaps. Structural layout diversity remains untested; evaluating it requires new training layouts while keeping evaluation layouts held out. Training on existing L1 test layouts would invalidate the split.

6 Conclusion

In this work, we introduced RoboFollow, a diagnostic benchmark for evaluating whether embodied agents genuinely follow linguistic instructions. RoboFollow combines high-entropy scene design, a hierarchical L0–L3 protocol, and decoupled intent-execution scoring to diagnose instruction following across spatial, attribute-based, procedural, and logical semantics. Under our fine-tuning setup, experiments on the evaluated VLA and WAM policies show that performance degrades sharply under minimal visual and semantic perturbations, and that existing optimization strategies remain insufficient. These results identify robust instruction following as a critical bottleneck for controllable and reliable embodied agents.

7 Limitations

RoboFollow is intended as a diagnostic benchmark rather than a comprehensive test of all embodied capabilities. Its controlled high-entropy scenes simplify object geometry, task horizons, and interaction dynamics to isolate instruction-following failures from execution confounds. Therefore, it does not fully cover long-horizon planning, contact-rich manipulation, open-vocabulary diversity, or large-scale real-world deployment. Our controls vary demonstration count, instruction variants, and training duration, but do not test increased structural layout diversity during training. The real-robot pilot compares different instruction sets, and we have not established transfer across simulators or to more complex real-world tasks. Future work should extend RoboFollow to richer embodiments and real-world scenarios while preserving its language-necessary design principle.

Acknowledgments

We thank the anonymous reviewers and the area chair for their constructive feedback and helpful suggestions. This work was supported in part by the Natural Science Foundation of China (Grant No.62503323).

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. *arXiv preprint arXiv:2312.13139*, 2023.
- [5] H. Bi, H. Tan, S. Xie, Z. Wang, S. Huang, H. Liu, R. Zhao, Y. Feng, C. Xiang, Y. Rong, et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.
- [6] Z. Feng, R. Xue, L. Yuan, Y. Yu, N. Ding, M. Liu, B. Gao, J. Sun, X. Zheng, and G. Wang. Multi-agent embodied ai: Advances and future directions. *arXiv preprint arXiv:2505.05108*, 2025.
- [7] K. Xu, Z. Zhu, A. Chen, S. Zhao, Q. Huang, Y. Yang, H. Lu, R. Xiong, M. Tomizuka, and Y. Wang. Seeing to act, prompting to specify: A bayesian factorization of vision language action policy. *arXiv preprint arXiv:2512.11218*, 2025.
- [8] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.
- [9] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [10] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [11] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

- [12] J. Zhang, X. Chen, Q. Wang, M. Li, Y. Guo, Y. Hu, J. Zhang, S. Bai, J. Lin, and J. Chen. Vlm4vla: Revisiting vision-language-models in vision-language-action models. *arXiv preprint arXiv:2601.03309*, 2026.
- [13] E. Collaboration, A. O’Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, A. Tung, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Gupta, A. Wang, A. Kolobov, A. Singh, A. Garg, A. Kembhavi, A. Xie, A. Brohan, A. Raffin, A. Sharma, A. Yavary, A. Jain, A. Balakrishna, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Wulfe, B. Ichter, C. Lu, C. Xu, C. Le, C. Finn, C. Wang, C. Xu, C. Chi, C. Huang, C. Chan, C. Agia, C. Pan, C. Fu, C. Devin, D. Xu, D. Morton, D. Driess, D. Chen, D. Pathak, D. Shah, D. Büchler, D. Jayaraman, D. Kalashnikov, D. Sadigh, E. Johns, E. Foster, F. Liu, F. Ceola, F. Xia, F. Zhao, F. V. Frujeri, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Feng, G. Schiavi, G. Berseth, G. Kahn, G. Yang, G. Wang, H. Su, H.-S. Fang, H. Shi, H. Bao, H. B. Amor, H. I. Christensen, H. Furuta, H. Bharadhwaj, H. Walke, H. Fang, H. Ha, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Abou-Chakra, J. Kim, J. Drake, J. Peters, J. Schneider, J. Hsu, J. Vakil, J. Bohg, J. Bingham, J. Wu, J. Gao, J. Hu, J. Wu, J. Wu, J. Sun, J. Luo, J. Gu, J. Tan, J. Oh, J. Wu, J. Lu, J. Yang, J. Malik, J. Silvério, J. Hejna, J. Booher, J. Tompson, J. Yang, J. Salvador, J. J. Lim, J. Han, K. Wang, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Black, K. Lin, K. Zhang, K. Ehsani, K. Lekkala, K. Ellis, K. Rana, K. Srinivasan, K. Fang, K. P. Singh, K.-H. Zeng, K. Hatch, K. Hsu, L. Itti, L. Y. Chen, L. Pinto, L. Fei-Fei, L. Tan, L. J. Fan, L. Ott, L. Lee, L. Weihs, M. Chen, M. Lepert, M. Memmel, M. Tomizuka, M. Itkina, M. G. Castro, M. Spero, M. Du, M. Ahn, M. C. Yip, M. Zhang, M. Ding, M. Heo, M. K. Srirama, M. Sharma, M. J. Kim, M. Z. Irshad, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. Liu, N. D. Palo, N. M. M. Shafiullah, O. Mees, O. Kroemer, O. Bastani, P. R. Sanketi, P. T. Miller, P. Yin, P. Wohlhart, P. Xu, P. D. Fagan, P. Mitrano, P. Sermanet, P. Abbeel, P. Sundareshan, Q. Chen, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Baijal, R. Scalise, R. Hendrix, R. Lin, R. Qian, R. Zhang, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Lin, S. Moore, S. Bahl, S. Dass, S. Sonawani, S. Tulsiani, S. Song, S. Xu, S. Haldar, S. Karamcheti, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Ramamoorthy, S. Dasari, S. Belkhale, S. Park, S. Nair, S. Mirchandani, T. Osa, T. Gupta, T. Harada, T. Matsushima, T. Xiao, T. Kollar, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, T. Chung, V. Jain, V. Kumar, V. Vanhoucke, V. Guizilini, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Chen, X. Wang, X. Zhu, X. Geng, X. Liu, X. Liangwei, X. Li, Y. Pang, Y. Lu, Y. J. Ma, Y. Kim, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Wu, Y. Xu, Y. Wang, Y. Bisk, Y. Dou, Y. Cho, Y. Lee, Y. Cui, Y. Cao, Y.-H. Wu, Y. Tang, Y. Zhu, Y. Zhang, Y. Jiang, Y. Li, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Ma, Z. Xu, Z. J. Cui, Z. Zhang, Z. Fu, and Z. Lin. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2025.
- [14] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2023.
- [15] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

- [16] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [17] X. Chen, Y. Chen, Y. Fu, N. Gao, J. Jia, W. Jin, H. Li, Y. Mu, J. Pang, Y. Qiao, Y. Tian, B. Wang, B. Wang, F. Wang, H. Wang, T. Wang, Z. Wang, X. Wei, C. Wu, S. Yang, J. Ye, J. Yu, J. Zeng, J. Zhang, J. Zhang, S. Zhang, F. Zheng, B. Zhou, and Y. Zhu. Internvla-m1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025.
- [18] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- [19] Z. Zhang, H. Xu, Z. Yang, C. Yue, Z. Lin, H. ang Gao, Z. Wang, and H. Zhao. Ta-vla: Elucidating the design space of torque-aware vision-language-action models. *arXiv preprint arXiv:2509.07962*, 2025.
- [20] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [21] Y. Xu, Y. Yang, Z. Fan, Y. Liu, Y. Li, B. Li, and Z. Zhang. Qvla: Not all channels are equal in vision-language-action model’s quantization. *arXiv preprint arXiv:2602.03782*, 2026.
- [22] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, A. Handa, M.-Y. Liu, D. Xiang, G. Wetzstein, and T.-Y. Lin. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. *arXiv preprint arXiv:2503.22020*, 2025.
- [23] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2024.
- [24] D. Wang, C. Liu, F. Chang, and Y. Xu. Hierarchical diffusion policy: manipulation trajectory generation via contact guidance. *arXiv preprint arXiv:2411.12982*, 2024.
- [25] H. Xue, J. Ren, W. Chen, G. Zhang, Y. Fang, G. Gu, H. Xu, and C. Lu. Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation. *arXiv preprint arXiv:2503.02881*, 2025.
- [26] S. Li, Y. Gao, D. Sadigh, and S. Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- [27] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, and J. Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- [28] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xi-ang, A. Malik, K. Lee, W. Liang, N. Ranawaka, J. Gu, Y. Xu, G. Wang, F. Hu, A. Narayan, J. Bjorck, J. Wang, G. Kim, D. Niu, R. Zheng, Y. Xie, J. Wu, Q. Wang, R. Julian, D. Xu, Y. Du, Y. Chebotar, S. Reed, J. Kautz, Y. Zhu, L. J. Fan, and J. Jang. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [29] B. Chen, T. Zhang, H. Geng, K. Song, C. Zhang, P. Li, W. T. Freeman, J. Malik, P. Abbeel, R. Tedrake, V. Sitzmann, and Y. Du. Large video planner enables generalizable robot control. *arXiv preprint arXiv:2512.15840*, 2025.

- [30] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas. *arXiv preprint arXiv:2512.15692*, 2025.
- [31] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, Y. Shen, and Y. Xu. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [32] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison. Rlbench: The robot learning benchmark & learning environment. *arXiv preprint arXiv:1909.12271*, 2019.
- [33] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- [34] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- [35] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan. Vima: General robot manipulation with multimodal prompts. *arXiv preprint arXiv:2210.03094*, 2023.
- [36] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, and X. Qiu. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv preprint arXiv:2412.18194*, 2024.
- [37] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, W. Deng, Y. Guo, T. Nian, X. Xie, Q. Chen, K. Su, T. Xu, G. Liu, M. Hu, H. ang Gao, K. Wang, Z. Liang, Y. Qin, X. Yang, P. Luo, and Y. Mu. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [38] Y. Mu, T. Chen, S. Peng, Z. Chen, Z. Gao, Y. Zou, L. Lin, Z. Xie, and P. Luo. Robotwin: Dual-arm robot benchmark with generative digital twins (early version). *arXiv preprint arXiv:2409.02920*, 2025.
- [39] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.
- [40] A. Yakefu, B. Xie, C. Xu, E. Zhang, E. Zhou, F. Jia, H. Yang, H. Fan, H. Zhang, H. Peng, J. Tan, J. Huang, K. Liu, K. Liu, K. Gu, Q. Zhang, R. Zhang, S. Huang, S. Cheng, S. Liu, T. Wang, T. Wang, W. Sun, W. Tang, Y. Wei, Y. Chen, Y. Gui, Y. Zhao, Y. Ma, Y. Wei, Y. Yang, Y. Guo, Z. Chen, Z. Du, Z. Zhang, Z. Liu, and Z. Yan. Robochallenge: Large-scale real-robot evaluation of embodied policies. *arXiv preprint arXiv:2510.17950*, 2025.
- [41] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- [42] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, et al. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [43] W.-H. Hsieh, E. Hsieh, D. Niu, T. Darrell, R. Herzig, and D. M. Chan. Do what? teaching vision-language-action models to reject the impossible. In *EMNLP (Findings)*, pages 11861–11869, 2025.
- [44] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

- [45] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [46] L. Zhong, Y. Liu, Y. Wei, Z. Xiong, M. Yao, S. Liu, and G. Ren. Acot-vla: Action chain-of-thought for vision-language-action models. *arXiv preprint arXiv:2601.11404*, 2026.
- [47] W. Wu, F. Lu, Y. Wang, S. Yang, S. Liu, F. Wang, Q. Zhu, H. Sun, Y. Wang, S. Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026.
- [48] T. Yuan, Z. Dong, Y. Liu, and H. Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [49] S. Lian, B. Yu, X. Lin, L. T. Yang, Z. Shen, C. Wu, Y. Miao, C. Huang, and K. Chen. Langforce: Bayesian decomposition of vision language action models via latent action queries. *arXiv preprint arXiv:2601.15197*, 2026.
- [50] Z. Zhan, Y. Chen, J. Zhou, Q. Lv, H. Liu, K. Wang, L. Lin, and G. Wang. Stable language guidance for vision-language-action models. *arXiv preprint arXiv:2601.04052*, 2026.

A Benchmark Details

RoboFollow is built upon the RoboTwin2.0 simulation platform[37]. We define four scenes and carefully partition the training data and evaluation tasks. In the training data, each task is provided with 50 episodes. When the relative object arrangements remain identical across episodes, a slight random positional perturbation of 1–2 cm is applied to each object. Furthermore, the instruction for each training episode is randomly sampled from a set of paraphrased variants to enrich semantic diversity during training. The template for the training instructions for scene1 is shown in Tab.4. The four scenes comprise a total of 3,750 training episodes. Our main experiments were all fine-tuned using all the data, while the analysis part of the experiments only used 16 tasks (800 episodes) from scene2 for fine-tuning.

Table 4: Scene 1 training instructions.

ID	Instruction Templates
Default scene	
1	T0: Use your left arm to pick up the block to the right of the red ball. T1: Use your left arm to grasp the block to the right of the red ball. T2: Pick up the block to the right of the red ball using your left arm.
2	T0: Use your right arm to pick up the block to the right of the red ball. T1: Use your right arm to grasp the block to the right of the red ball. T2: Pick up the block to the right of the red ball using your right arm.
3	T0: Use your left arm to pick up the block to the left of the green ball. T1: Use your left arm to grasp the block to the left of the green ball. T2: Pick up the block to the left of the green ball using your left arm.
4	T0: Use your right arm to pick up the block in front of the red cylinder. T1: Use your right arm to grasp the block in front of the red cylinder. T2: Pick up the block in front of the red cylinder using your right arm.
5	T0: Use your right arm to pick up the block to the right of the red ball and place it behind the red cylinder. T1: Use your right arm to grasp the block to the right of the red ball and put it behind the red cylinder. T2: Pick up the block to the right of the red ball and place it behind the red cylinder using your right arm.
6	T0: Use your right arm to pick up the block to the right of the red ball and place it to the right of the red cylinder. T1: Use your right arm to grasp the block to the right of the red ball and put it to the right of the red cylinder. T2: Pick up the block to the right of the red ball and place it on the right side of the red cylinder using your right arm.
7	T0: Use your left arm to pick up the block to the right of the red ball and place it behind the red ball. T1: Use your left arm to grasp the block to the right of the red ball and put it behind the red ball. T2: Pick up the block to the right of the red ball and place it behind the red ball using your left arm.
8	T0: Use your left arm to pick up the block to the right of the red ball and place it to the left of the red ball. T1: Use your left arm to grasp the block to the right of the red ball and put it to the left of the red ball. T2: Pick up the block to the right of the red ball and place it on the left side of the red ball using your left arm.
9	T0: Use your left arm to pick up the block to the left of the green ball and place it behind the green ball. T1: Use your left arm to grasp the block to the left of the green ball and put it behind the green ball. T2: Pick up the block to the left of the green ball and place it behind the green ball using your left arm.

Continued on next page

ID	Instruction Templates
10	T0: Use your left arm to pick up the block to the left of the green ball and place it in front of the green ball. T1: Use your left arm to grasp the block to the left of the green ball and put it in front of the green ball. T2: Pick up the block to the left of the green ball and place it in front of the green ball using your left arm.
11	T0: Use your left arm to pick up the block to the left of the green ball and place it behind the red ball. T1: Use your left arm to grasp the block to the left of the green ball and put it behind the red ball. T2: Pick up the block to the left of the green ball and place it behind the red ball using your left arm.
12	T0: Use your left arm to pick up the block to the left of the green ball and place it to the left of the red ball. T1: Use your left arm to grasp the block to the left of the green ball and put it to the left of the red ball. T2: Pick up the block to the left of the green ball and place it on the left side of the red ball using your left arm.
13	T0: Use your right arm to pick up the block in front of the red cylinder and place it behind the red cylinder. T1: Use your right arm to grasp the block in front of the red cylinder and put it behind the red cylinder. T2: Pick up the block in front of the red cylinder and place it behind the red cylinder using your right arm.
14	T0: Use your right arm to pick up the block in front of the red cylinder and place it to the right of the red cylinder. T1: Use your right arm to grasp the block in front of the red cylinder and put it to the right of the red cylinder. T2: Pick up the block in front of the red cylinder and place it on the right side of the red cylinder using your right arm.
15	T0: Use your right arm to pick up the block in front of the red cylinder and place it behind the green ball. T1: Use your right arm to grasp the block in front of the red cylinder and put it behind the green ball. T2: Pick up the block in front of the red cylinder and place it behind the green ball using your right arm.
16	T0: Use your right arm to pick up the block in front of the red cylinder and place it in front of the green ball. T1: Use your right arm to grasp the block in front of the red cylinder and put it in front of the green ball. T2: Pick up the block in front of the red cylinder and place it in front of the green ball using your right arm.

The following are the specific training and testing task designs for the four scenarios:

Scene 1: Extrinsic Spatial Relations.

Variants of the Scene 1 Layout are shown in Tab.5:

Table 5: Scene 1 Layout Variants from the Default Scene

Scene 1.1	Swaps the green ball and the red cylinder, and then swaps the red cylinder and the red ball.
Scene 1.2	Swaps the red ball and the green ball.
Scene 1.3	Swaps the green ball and the red cylinder.

In this example shown in Fig. 2, the scene–instruction pair in L0 appears in the training set. In L1, the instruction from L0 is retained, but the scene layout is changed, where the positions of the red ball and the green ball are swapped. In this setting, we expect the model to preserve the correct understanding of the phrase “to the right of the red ball”. Specifically, the model should grasp the block located to the right of the relocated red ball, rather than relying on memorized coordinates

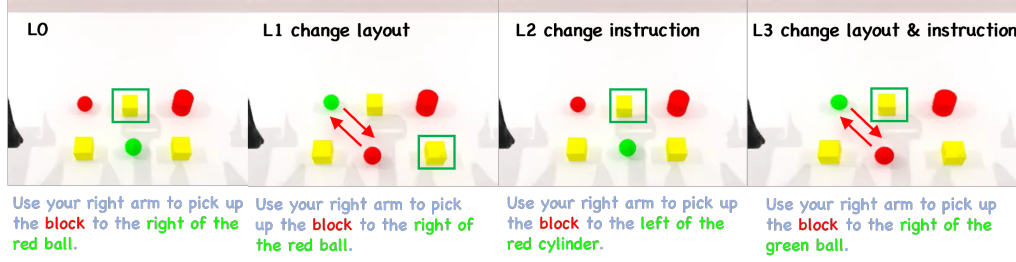


Figure 2: Example of Scene 1.

from the L0 training examples and incorrectly associating “to the right of the red ball” with a fixed spatial position (e.g., the upper-center region of the image).

In L2, the scene layout remains the same as the training-time default scene. The target block is still located at the upper-center position, but the instruction given to the model is changed to “the block to the left of the red cylinder.” This setting tests whether the model can correctly interpret a novel compositional instruction. Although the individual concepts such as left and red cylinder appear in the training data, their combination does not.

Finally, L3 simultaneously changes both the scene layout and the instruction. In this case, the layout is switched, and the instruction becomes “the block to the right of the green ball.” Similar to L2, the individual concepts (right, green ball) have appeared in the training data, but their composition has not.

Importantly, the target actions in all these cases are present in the training set. Therefore, any failure cannot be attributed to the model encountering unseen actions, but rather reflects its ability to generalize across novel scene configurations and compositional instructions.

Scene 2: Intrinsic Object Properties.

Variants of the Scene 2 Layout are shown in Tab.6:

Table 6: Scene 2 Layout Variants from the Default Scene

Scene 2.1	Swaps the large red cylinder and the large blue cube.
Scene 2.2	Swaps the large blue cube and the small blue cylinder.
Scene 2.3	Swaps the small red cube and the small blue cylinder.
Scene 2.4	Swaps the large red cylinder and the small red cube.
Scene 2.5	Swaps the green bowl and the yellow bowl.
Scene 2.6	Swaps the small red cube and the small blue cylinder, and also swaps the green bowl and the yellow bowl.

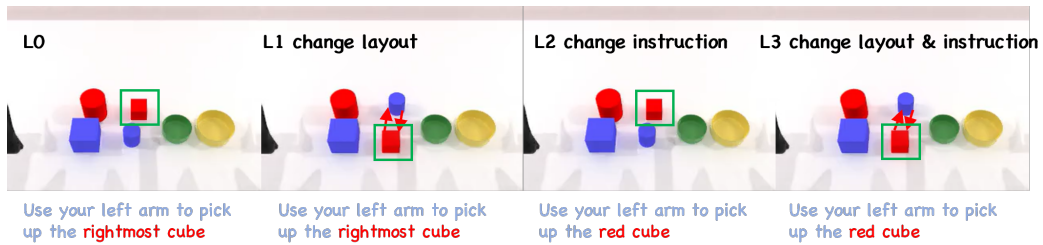


Figure 3: Example of Scene 2.

In this example shown in Fig. 3, L1 retains the instruction from L0, but the scene layout is modified by swapping the positions of the blue cylinder and the red cube. Under this setting, we expect the model to preserve the semantic understanding of “rightmost cube” and correctly grasp the red cube at its new rightmost position, rather than memorizing the specific coordinates observed during training and associating “rightmost cube” with a fixed location in the training scene.

In L2, the scene layout remains the default scene observed during training, where the target object is still the small red cube. However, the instruction given to the model is modified to refer explicitly to the “red cube”. This setting evaluates whether the model can correctly interpret the new instruction. Notably, the concepts “red” and “cube” both appear in the training data, but the specific compositional phrase “red cube” does not, allowing us to test the model’s ability to generalize compositionally.

Finally, L3 introduces changes to both the scene layout and the instruction. Importantly, the target actions in all these settings appear in the training dataset, ensuring that any execution failures cannot be attributed to the model encountering previously unseen actions, but rather reflect its capability in instruction understanding and generalization.

Scene 3: Fine grained Action Modulation and Trajectory Constraints.

Variants of the Scene 3 Layout are shown in Tab.7:

Table 7: Scene 3 Layout Variants from the Default Scene

Scene 3.1	Swaps the green slab and the red slab.
Scene 3.2	Swaps the blue slab and the yellow slab.
Scene 3.3	Swaps the red slab and the yellow slab.
Scene 3.4	Swaps the green slab and the blue slab.

In this example shown in Fig. 4, although L1 retains the same instruction as L0, the scene layout is modified by swapping the positions of the green slab and the red slab. Under this change, we expect the model to preserve its correct interpretation of the constraint “avoid passing over the red slab”. Specifically, the model should move the yellow slab along a trajectory that passes over the green slab, while still avoiding the red slab in the new location.

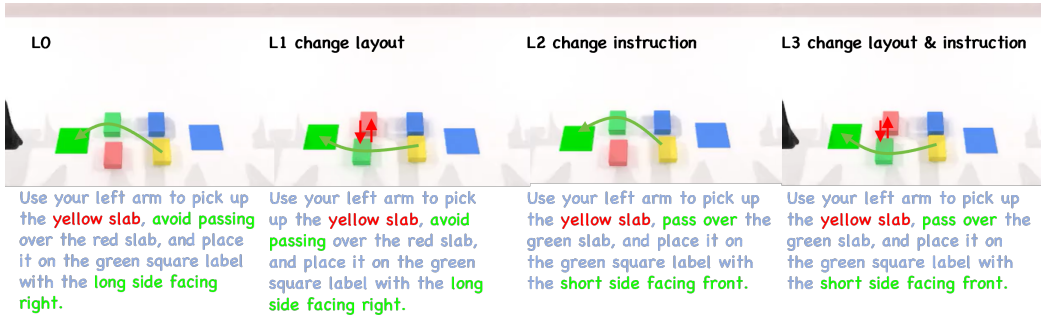


Figure 4: Example of Scene 3.

In L2, the scene retains the default training layout, while the instruction is reformulated. In this example, “avoid passing over the red slab” is replaced with “pass over the green slab”, and “long side facing right” is replaced with “short side facing front”. Under the benchmark’s paired-route convention, the route expressions specify the same intended route, and the orientation expressions describe the same final placement orientation. Thus, L2 changes the linguistic formulation while preserving the intended behavior. All constituent concepts are present in the training instructions.

Finally, L3 simultaneously modifies both the scene layout and the instruction, combining the changes introduced in L1 and L2.

Scene 4: Elementary Logical Grounding.

Variants of the Scene 4 Layout are shown in Tab.8:

Table 8: Scene 4 Layout Variants from the Default Scene

Scene 4.1	Removes the cube inside the bowl and swaps the blue sphere and the red sphere.
Scene 4.2	Removes the cube inside the bowl, places the blue cube on top of the yellow cylinder, moves the green cylinder down to the tabletop at the original blue-cube position, and moves the red sphere to the right side
Scene 4.3	Removes the cube inside the bowl.
Scene 4.4	Swaps the blue sphere and the red sphere.
Scene 4.5	Places the green cylinder on top of the yellow cylinder.
Scene 4.6	Swaps the positions of the blue cube (together with the green cylinder stacked on it) and the yellow cylinder.
Scene 4.7	Moves the red sphere to the right side.

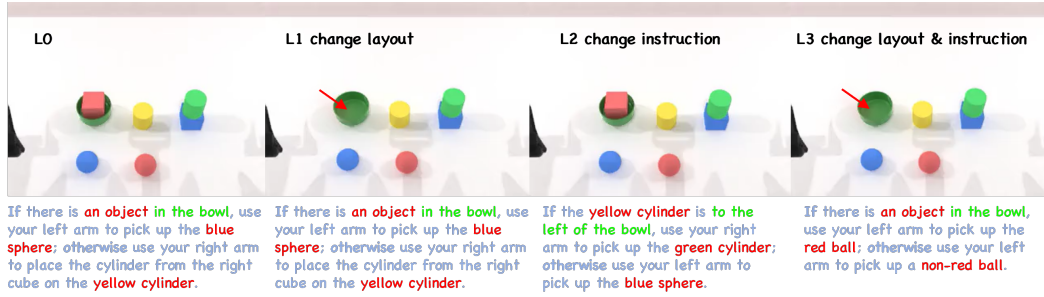


Figure 5: Example of Scene 4.

In this example shown in Fig. 5, L1 retains the same instruction as L0, but the scene layout is modified: the object inside the bowl is removed. Under this change, we expect the model to correctly interpret the semantic condition and execute the otherwise branch of the instruction.

In L2, the scene layout remains the default scene observed during training, while the instruction provided to the model is altered by modifying the condition. This setting evaluates whether the model can correctly understand and follow the updated instruction.

Finally, L3 introduces changes to both the scene layout and the instructional execution branch, simultaneously testing the model’s ability to generalize across variations in both environmental configuration and instruction semantics.

A.1 Scene Entropy Computation

Group training tasks by their task-independent initial scene specification: fixtures, visible objects and attributes, placement distributions, and observable initial-state predicates. Exclude task names, language, and goal annotations; random jitters from a shared distribution and paraphrases do not create new scene groups or task labels. Scene 1/2/3 each has one group of 16 task labels; Scene 4 has three groups of 16, 7, and 4. With 50 demonstrations per task,

$$H_{\text{scene}} = \frac{64}{75} \log_2 16 + \frac{7}{75} \log_2 7 + \frac{4}{75} \log_2 4 = 3.782 \text{ bits.}$$

For example, a 16-task group contributes $(16/75) \log_2 16$ bits. The corresponding LIBERO Spatial/Object/Goal/Long values are 0/0/3.322/0.200 bits, whose equally weighted mean is 0.880 bits.

B Metrics Details

We evaluate each episode using a stage-based protocol rather than a single binary success label. Across scenes, a task is decomposed into up to three functionally defined stages. Stage 1 evaluates initial source-object grounding (or initial contact for push actions), Stage 2 evaluates the action-specific manipulation objective, and Stage 3 evaluates the post-manipulation outcome, such as retraction quality or final placement pose. For pickup-only tasks that do not involve an intermediate manipulation objective, Stage 2 is omitted. This design allows the evaluator to capture whether a policy fails at early grounding, during manipulation, or after the main action has nominally been completed.

For each applicable stage, we report two complementary metrics. Intent Score measures whether the policy grounds the instruction to the correct object, relation, or logical branch and moves in the semantically appropriate direction, even when the physical execution is imperfect. Execution Score measures whether the corresponding physical sub-goal is successfully completed. This separation is intended to make semantic misgrounding and execution failure more distinguishable than in conventional final-state success evaluation. As a result, approaching the correct target without completing the manipulation can still receive partial semantic credit, while interacting with an incorrect object is penalized even if the final state happens to look plausible.

The stage definitions follow a shared template across scenes while remaining aligned with the semantic focus of each scenario. In Scenes 1 and 2, the evaluation focuses on correct source selection, completion of the specified action objective, and post-action stabilization or retraction. In Scene 3, the intermediate stage focuses on compliance with trajectory constraints, and the final stage evaluates placement quality and orientation. In Scene 4, the same framework is extended to instructions involving negation, conditionals, and temporal order, with stage definitions adapted to standard, conditional, and sequential tasks.

CR uses action-dependent criteria: pickup-like tasks require both final IS and ES to reach 0.8; other tasks use target-layout or action-progress signals, with designated execution-stage scores as fallbacks. In Scenes 1, 2, and 4, `stage3_finish` is not checked separately by CR, but its credit remains in the totals used for pickup-like tasks. Scene 3 uses positive execution credit in its final placement/orientation stage. CR does not require full credit at every intermediate stage, and subsequent actions can affect CR by changing the evaluated final state.

C Policies Training Details

In the main experiments, each policy was fine-tuned on the full set of 3,750 training episodes. Training steps were selected per model based on in-distribution convergence, ensuring each policy reached its performance plateau before evaluation.

Table 9 summarizes the batch size and training duration for each policy.

Table 9: Training hyperparameters for each policy. All models are fine-tuned on the full set of 3,750 training episodes. Training steps are selected per model based on in-distribution (L0) convergence.

Model	Batch Size	Steps
$\pi_{0.5}$	32	20,000
π_0	32	30,000
GR00T N1.6	64	32,000
Motus	32	8,000
OpenVLA-OFT	32	30,000
XVLA	32	30,000
ACoT-VLA	16	20,000
Lingbot-VLA	32	30,000
FAST-WAM	64	15,000

C.1 Real-Robot Pilot Details

We use $\pi_{0.5}$, all instructions specify the right arm. The eight training instructions and eight held-out instructions are each evaluated in five trials, giving 40 trials per set and 80 trials in total. These counts refer to evaluation rollouts, not training demonstrations or independent model-training runs. Table 10 lists both instruction sets and their per-instruction success counts. Figure 6 shows the experimental setup and representative real-robot executions.

Table 10: Real-robot instructions and success counts. Each instruction has five evaluation trials. IDs are local to each set and do not indicate matched pairs.

ID	Instruction	Successes/trials
Training instructions		
1	Use your right arm to pick up the larger red object.	5/5
2	Use your right arm to pick up the leftmost cube.	3/5
3	Use your right arm to pick up the rightmost cube.	5/5
4	Use your right arm to pick up the blue cylinder.	1/5
5	Use your right arm to stack the leftmost cube onto the larger red object.	2/5
6	Use your right arm to stack the leftmost cube onto the smaller blue object.	3/5
7	Use your right arm to stack the blue cylinder onto the leftmost cube.	0/5
8	Use your right arm to stack the blue cylinder onto the rightmost cube.	1/5
Held-out instructions		
1	Use your right arm to pick up the blue cube.	4/5
2	Use your right arm to pick up the larger blue object.	2/5
3	Use your right arm to pick up the larger cylinder.	0/5
4	Use your right arm to pick up the red cube.	0/5
5	Use your right arm to pick up the smaller cube.	0/5
6	Use your right arm to pick up the smaller red object.	0/5
7	Use your right arm to stack the blue cylinder onto the blue cube.	0/5
8	Use your right arm to place the blue cylinder on top of the red cube.	0/5

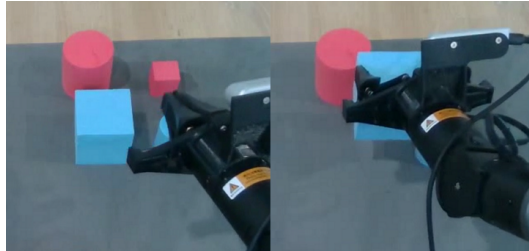


Figure 6: Real-robot experimental setup and representative executions of $\pi_{0.5}$ with the right arm.

Success is 20/40 (50%) on training instructions and 6/40 (15%) on held-out instructions, a descriptive difference of 35 percentage points. Because every instruction has five trials, the pooled rate equals the unweighted mean of instruction-level rates within each set.

The training set contains four pick and four stack instructions, while the held-out set contains six pick and two stack instructions. Within these categories, success is 14/20 (70%) versus 6/30 (20%) for pick instructions, and 6/20 (30%) versus 0/10 (0%) for stack instructions. These are descriptive comparisons of different instruction sets; neither the pooled contrast nor the category breakdown controls for task identity or difficulty.

In a rollout for held-out instruction 4, the policy attempted to pick the blue cube instead of the requested red cube. This is qualitative evidence of a grounding error, not a quantified Intent Score.

Under a pooled independent-binomial approximation, the 95% Wilson intervals for 20/40 and 6/40 are [35.2, 64.8]% and [7.1, 29.1]%, respectively. These approximate intervals do not account for

instruction heterogeneity or dependence between trials and do not measure variation across independently trained policies.

D QA-20 details

This section provides the complete set of visual question-answering (VQA) questions referenced in the main text, listed in Table 11.

Table 11: Scene 2 visual question-answering questions.

ID	Questions
Default scene	
1	What is the object behind the blue cube?
2	Output the color of the object to the right of the green bowl.
3	Output the color of the leftmost bowl.
4	Output the color of the rightmost cube.
5	Output the color of the cylinder to the left of the red cube.
6	Output the color of the object in front of the red cube.
7	Describe the object to the right of the green bowl, such as what it is and what color it is.
8	What are the red objects in the scene, and what are their positions?
9	Which object is located on the right side of the blue cube?
10	Output the color of the rightmost cylinder.
11	Looking at the two red objects on the table, which one is on the left and which one is on the right?
12	Output the color of the object on the right side of the red cylinder.
13	Output the color of the leftmost cylinder.
14	Where are the two bowls located relative to each other? Which bowl is on the left side?
15	What is the object behind the blue cylinder?
16	Output the color of the object in front of the red cylinder.
17	Output the color of the object to the right of the red cylinder.
18	Output the color of the object to the left of the blue cylinder.
19	Output the color of the object to the left of the yellow bowl.
20	Output the color of the object to the right of the blue cylinder.

E Failure Cases

As shown in Fig. 7, the failures fall into two categories. The first occurs when the model understands the task intent but fails due to insufficient execution precision, and the second stems from weaknesses in semantic reasoning and instruction grounding, such as misunderstanding spatial relationships, failing to compare or compose object attributes, inability to meet fine-grained trajectory or orientation constraints, and difficulty handling logical constructs like conditionals, temporal order, and negation. Additionally, the policy may fail to terminate after completing the task, continuing unintended actions not specified in the instruction.

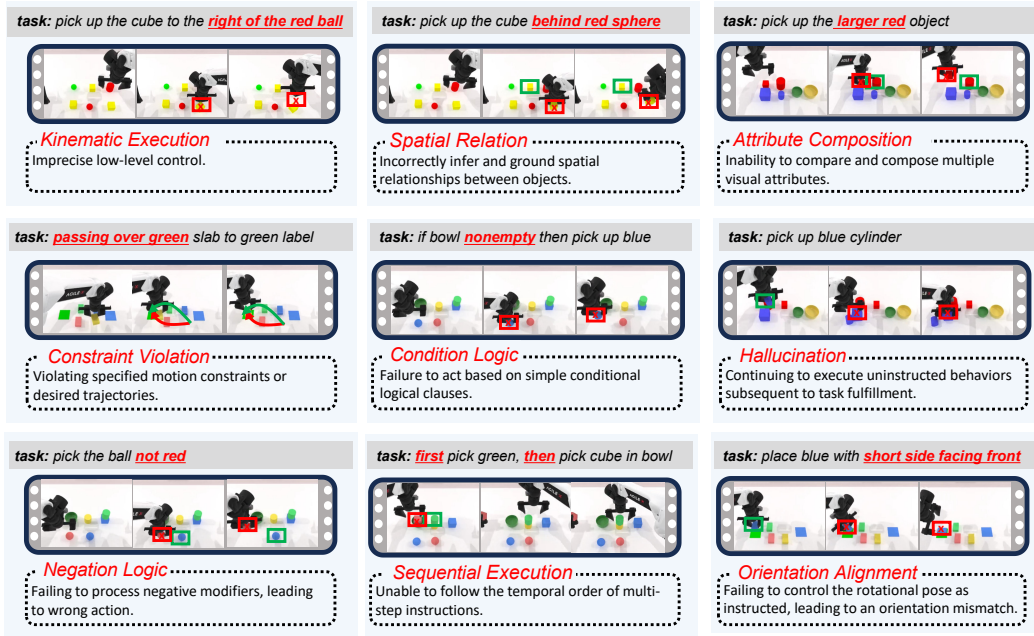


Figure 7: Representative failure cases of the policy.